

CI/CD PIPELINE DESIGN DOCUMENT

Case Study: Online Food Delivery Platform

(Microservices-based Web & Mobile Application)

Course Code / Title:	CSA10 — Software Engineering
CO Assessed:	CO4 — Robust Testing, Quality Assurance & DevOps Practices
Assessment:	Assessment Tool 2 — CI/CD Pipeline Design Exercise
Weightage / Marks:	20% / 25 Marks
Student Name:	R. Indhu
Register Number:	192524332

Table of Contents

1. Introduction & Problem Statement	3
2. Requirement Analysis & CI/CD Needs	3
3. Pipeline Architecture & Workflow	4
4. Pipeline Stages Explained	4
5. Tools & Technology Selection	5
6. Automated Testing Strategy.....	6
7. Deployment Strategy.....	6
8. Security, Quality Checks, Notifications & Rollback.....	7
9. CI/CD Workflow: Commit to Deployment (Flowchart)	7
10. Sample Pipeline Configuration	9
11. Conclusion.....	9

1. Introduction & Problem Statement

The assigned problem statement concerns an Online Food Delivery Platform, a microservices-based system comprising a customer-facing web and mobile application, a restaurant partner portal, a delivery-partner tracking service, a payments module, and an order-management backend. The system experiences frequent feature releases, seasonal traffic spikes (festivals, weekends), and a strict requirement of near-zero downtime because customers and restaurant partners depend on the platform continuously for order placement, live tracking, and payment processing.

Because multiple development teams work on independent microservices (User Service, Order Service, Payment Service, Delivery-Tracking Service, and Notification Service) in parallel, manual build-test-deploy cycles are error-prone, slow, and difficult to audit. A well-designed CI/CD (Continuous Integration / Continuous Deployment) pipeline is therefore essential to automate integration, testing, quality checks, and deployment while enforcing security and reliability at every stage, directly addressing CO4: robust testing, quality assurance, and DevOps practices with an emphasis on security, reliability, and ethics.

1.1 Objectives of the CI/CD Pipeline

- **Continuous Integration:** Automatically build and validate every code change committed by any microservice team.
- **Early Defect Detection:** Run automated unit, integration, and regression tests on every commit to catch defects early.
- **Quality & Security Gates:** Enforce code quality, security, and vulnerability scanning before code is packaged.
- **Automated Deployment:** Deploy validated builds to Development, Staging, and Production environments with minimal manual effort.
- **Reliability & Rollback:** Support instant rollback and alerting when a deployment fails or degrades service health.

2. Requirement Analysis & CI/CD Needs

Analyzing the food-delivery platform's architecture and business constraints surfaces the following CI/CD requirements, mapped against the risk they mitigate:

Requirement	Description	Risk Mitigated
Automated Source Control Integration	Every microservice repository must trigger a build automatically on push/PR merge.	Delayed or missed integration of changes
Fast, Isolated Builds	Each microservice builds and tests independently using containerized build agents.	Cross-service build failures blocking unrelated teams
Comprehensive Automated Testing	Unit, integration, contract, and load tests run automatically per service.	Undetected functional and performance regressions
Code Quality & Security Gates	Static analysis and dependency/vulnerability scans block insecure or low-quality code.	Security breaches, technical debt
Multi-Environment Promotion	Builds promote through Dev to Staging to Production with controlled gates.	Untested code reaching real customers
Zero-Downtime Deployment	Blue-Green / Canary deployment for the order and payment services.	Service outage during peak ordering hours
Monitoring, Alerting & Rollback	Real-time health checks trigger notifications and automatic rollback.	Prolonged outages and unnoticed failures

3. Pipeline Architecture & Workflow

The proposed pipeline follows a linear seven-stage architecture, beginning at source-code commit and ending in a production deployment, with automated feedback loops for failures and automated rollback on unhealthy deployments. Figure 1 illustrates the complete architecture.

Figure 1: End-to-End CI/CD Pipeline Architecture

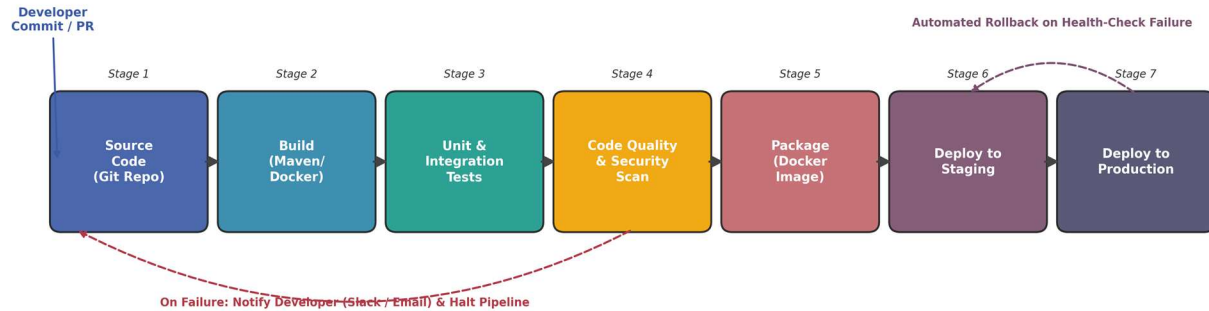


Figure 1: End-to-End CI/CD Pipeline Architecture for the Food Delivery Platform

Each microservice (User, Order, Payment, Delivery-Tracking, Notification) passes through this same pipeline independently, allowing teams to release features without waiting for unrelated services. The pipeline is triggered automatically by a webhook on every commit or pull-request merge into the main integration branch.

4. Pipeline Stages Explained

Stage	Purpose	Key Activities
1. Source Control	Version-controlled, collaborative code management	Feature branching, pull-request review, branch-protection rules, mandatory code review before merge
2. Build	Compile and containerize the application	Dependency resolution, compilation, Docker image build, artifact versioning/tagging
3. Automated Testing	Verify functional correctness	Unit tests, integration tests, service-contract tests, code-coverage threshold enforcement ($\geq 80\%$)
4. Code Quality & Security Scan	Enforce maintainability and security	Static analysis (SonarQube), dependency vulnerability scan (Snyk/OWASP), secrets detection
5. Packaging	Produce a deployable, immutable artifact	Docker image build, image signing, push to private container registry

Stage	Purpose	Key Activities
6. Deploy to Staging	Validate in a production-like environment	Automated deployment, smoke tests, performance/load testing, UAT sign-off
7. Deploy to Production	Release to end users safely	Blue-Green/Canary rollout, health-check verification, monitoring activation

5. Tools & Technology Selection

The following tools were selected for each stage, chosen for their strong community support, native container/microservice integration, and suitability for the platform's Java/Node.js microservice stack:

Pipeline Stage	Tool(s) Selected	Justification
Source Control	GitHub / GitLab	Industry-standard hosting with built-in pull-request review, branch protection, and webhook triggers for CI.
CI Orchestration	Jenkins / GitHub Actions	Mature ecosystem, wide plugin support for Docker, Kubernetes, and notification integrations; supports declarative pipeline-as-code.
Build & Packaging	Maven / npm + Docker	Maven and npm handle language-specific dependency resolution; Docker guarantees environment parity across Dev, Staging, and Production.
Automated Testing	JUnit / Jest, Postman/Newman, JMeter	JUnit/Jest cover unit tests; Postman/Newman validate REST contracts between microservices; JMeter simulates peak-hour load.
Code Quality	SonarQube	Provides code-smell, duplication, and maintainability metrics with a configurable quality gate that can fail the build.
Security Scanning	OWASP Dependency-Check, Snyk	Detects known CVEs in third-party libraries and container base images before deployment.

Pipeline Stage	Tool(s) Selected	Justification
Container Registry	Docker Hub / Amazon ECR	Central, versioned storage of deployable images with access control and image-scanning support.
Deployment Orchestration	Kubernetes + Helm	Enables declarative, repeatable Blue-Green/Canary deployments and horizontal auto-scaling for traffic spikes.
Monitoring & Logging	Prometheus, Grafana, ELK Stack	Real-time metrics, dashboards, and centralized log aggregation for rapid fault diagnosis.
Notifications	Slack / Microsoft Teams + Email	Immediate alerting of build/test/deployment failures to the responsible microservice team.

6. Automated Testing Strategy

A layered ('testing pyramid') strategy is integrated directly into the pipeline so that fast, cheap tests run first and slower, expensive tests run only after earlier layers pass:

Test Layer	Scope	Tool(s)	Pipeline Gate
Unit Tests	Individual functions/classes within a microservice	JUnit, Jest, Mockito	Runs on every commit; blocks merge on failure
Integration Tests	Interaction between a service and its database/queue	Testcontainers, Spring Test	Runs after unit tests in the build stage
Contract / API Tests	Inter-service REST/message contracts	Postman/Newman, Pact	Runs before staging deployment
Regression & Smoke Tests	Critical user journeys (order → payment → tracking)	Selenium, Cypress	Runs automatically after staging deployment

Test Layer	Scope	Tool(s)	Pipeline Gate
Performance/Load Tests	Response time & throughput under peak order volume	JMeter, k6	Runs in Staging before production promotion
Security Tests	Dependency CVEs, static code vulnerabilities	OWASP ZAP, Snyk	Runs in the quality-gate stage; blocks packaging on critical findings

A minimum code-coverage threshold of 80% and a zero-tolerance policy for critical/high-severity vulnerabilities are enforced as automated quality gates; failing either halts the pipeline and notifies the owning team, preventing untested or insecure code from reaching staging.

7. Deployment Strategy

Three progressively stricter environments are used to balance development speed with production safety, as shown in Figure 2.

Figure 2: Multi-Environment Deployment Strategy

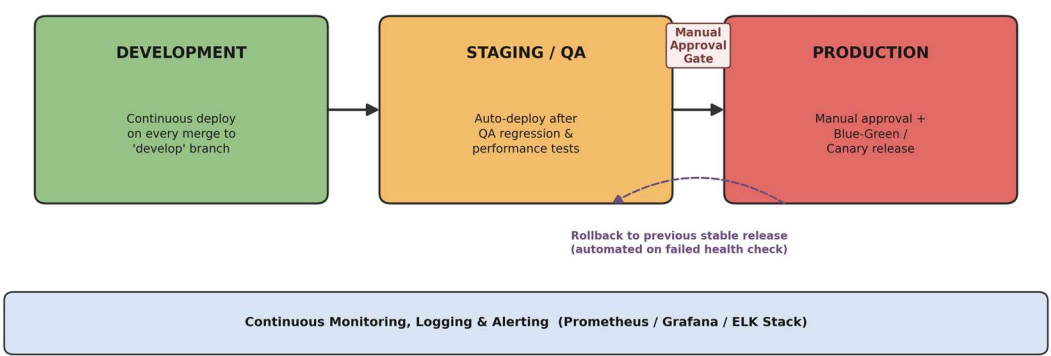


Figure 2: Multi-Environment Deployment Strategy with Approval Gate and Rollback

Environment	Purpose	Deployment Trigger
Development	Continuous integration testing for developers; unstable, frequently updated	Automatic on every merge to the 'develop' branch
Staging / QA	Production-like environment for QA sign-	Automatic after successful build & quality gates

Environment	Purpose	Deployment Trigger
	off, performance and UAT testing	
Production	Live environment serving real customers, restaurants, and delivery partners	Manual approval, followed by automated Blue-Green/Canary rollout

For Production, the Payment and Order services use Canary releases (routing a small percentage of live traffic to the new version before a full rollout) while less critical services such as Notification use Blue-Green deployment for simplicity. Both strategies allow near-instant rollback by redirecting traffic back to the previous stable version.

8. Security, Quality Checks, Notifications & Rollback Mechanisms

8.1 Security Practices

- **Shift-Left Security:** Static and dependency scans (SonarQube, OWASP Dependency-Check, Snyk) run automatically before packaging; builds with critical vulnerabilities are blocked.
- **Secrets Management:** API keys, database credentials, and payment-gateway secrets are stored in a dedicated secrets manager (e.g., HashiCorp Vault) and never committed to source control.
- **Image Integrity:** Only signed, scanned container images from the private registry are permitted to deploy, verified via an image-admission policy in Kubernetes.
- **Access Control & Auditability:** All production deployments require role-based manual approval, and every pipeline run is logged for audit and compliance review.

8.2 Notifications

Slack/Teams channels and email alerts are configured for every pipeline stage transition. Build failures, failed test suites, quality-gate rejections, and deployment failures immediately notify the owning microservice team, while successful production deployments notify the wider engineering channel for visibility.

8.3 Rollback Mechanism

Post-deployment health checks (HTTP readiness/liveness probes, error-rate and latency thresholds in Prometheus) continuously validate the new release. If thresholds are breached

within a defined observation window, Kubernetes/Helm automatically redirects traffic back to the last known-good image version without manual intervention, and an incident alert is raised for the on-call engineer.

9. CI/CD Workflow: Commit to Deployment

Figure 3 presents the complete workflow as a flowchart, tracing a single code change from developer commit through to a monitored production release, including the decision points and failure/notification paths described above.

Figure 3: CI/CD Workflow Flowchart — Commit to Deployment



Figure 3: CI/CD Workflow Flowchart — Commit to Deployment

10. Sample Pipeline Configuration (Illustrative)

A simplified GitHub Actions-style configuration illustrating how the Order Service stage sequence in Figure 1 maps to pipeline-as-code:

```
name: order-service-ci-cd
on:
  push:
    branches: [ develop, main ]
jobs:
  build-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build with Maven
        run: mvn -B clean package
      - name: Run Unit & Integration Tests
        run: mvn test
      - name: SonarQube Quality Gate
        run: mvn sonar:sonar
      - name: Dependency Vulnerability Scan
        run: snyk test --severity-threshold=high
  package-deploy-staging:
    needs: build-test
    runs-on: ubuntu-latest
    steps:
      - name: Build & Push Docker Image
        run: docker build -t registry/order-service:$GITHUB_SHA .
      - name: Deploy to Staging (Helm)
        run: helm upgrade order-service ./chart --namespace staging
      - name: Run Smoke Tests
        run: newman run staging-smoke-tests.json
  deploy-production:
    needs: package-deploy-staging
    environment: production # requires manual approval
    runs-on: ubuntu-latest
    steps:
      - name: Canary Deploy to Production (Helm)
```

```
run: helm upgrade order-service ./chart -n production --set canary=true  
- name: Verify Health & Auto-Rollback  
run: ./scripts/verify-health-and-rollback.sh
```

11. Conclusion

The proposed CI/CD pipeline automates the Online Food Delivery Platform's journey from code commit to production release across seven well-defined stages, integrating automated testing, static and dependency security scanning, and a multi-environment deployment strategy with manual production approval. Blue-Green and Canary release techniques, combined with continuous health monitoring and automated rollback, ensure high reliability and near-zero downtime despite frequent releases and variable traffic. Centralized notifications keep every microservice team informed of failures in real time, while audit logging and secrets management uphold security and ethical handling of sensitive payment and customer data. Together, these practices fulfil CO4's emphasis on robust testing, quality assurance, and DevOps best practices.

References

- Humble, J. & Farley, D., Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation, Addison-Wesley.
- Jenkins Official Documentation — <https://www.jenkins.io/doc/>
- Kubernetes Official Documentation — <https://kubernetes.io/docs/>
- SonarQube Documentation — <https://docs.sonarsource.com/>
- OWASP Dependency-Check — <https://owasp.org/www-project-dependency-check/>